Informe Tarea 1: Taller de Integración TIC

Adquisición y Telemetría del Sensor BMI270 con ESP32-S3 y Raspberry Pi 5

Integrantes: Benjamín Barrueto, Benjamín Pacheco

Profesor: Luciano Radriagán – Asignatura: TIC3 – 25 Octubre de 2026

---

1 Introducción y Arquitectura del Sistema

El presente informe describe el desarrollo y las pruebas de un sistema de adquisición de datos implementado para la Tarea 1 del curso Taller de Integración TIC. El objetivo del proyecto consiste en leer en tiempo real las mediciones de aceleración lineal y velocidad angular de un sensor BMI270 conectado a un microcontrolador ESP32-S3, y transmitirlas de manera inalámbrica por Wi-Fi hacia una Raspberry Pi 5. En la estación base se dispone de una interfaz gráfica que permite configurar los parámetros de captura, visualizar las señales en vivo y almacenar las sesiones en archivos CSV para su posterior análisis.

La arquitectura del sistema sigue un modelo cliente-servidor donde la Raspberry Pi 5 actúa como servidor escuchando en el puerto 5000, mientras que el ESP32-S3 funciona como cliente conectándose a la dirección IP de la Raspberry Pi. Esta organización permite que el microcontrolador no tenga valores de captura fijos en su código, sino que espere las instrucciones enviadas por el usuario desde la interfaz gráfica antes de comenzar cada medición.

Para mantener un diseño ordenado y modular, el software se dividió en tres partes principales, las cuales se resumen en la Tabla 1.

Tabla 1: Organización general de los módulos del sistema.

Componente	Archivo Principal	Función dentro del Sistema
Lectura de sensor	main.c	Configura el bus I2C del ESP32-S3, inicializa el sensor BMI270, ajusta la frecuencia y el rango antes de cada sesión y convierte las lecturas a unidades físicas.
Cliente de red	tcp_cliente.c	Conecta el ESP32-S3 a la red Wi-Fi, recibe los comandos de inicio y detención por TCP y envía las muestras en formato JSON mediante TCP o UDP.
Servidor e interfaz	codigoraspberry.py	Recibe los datos en la Raspberry Pi 5 sin bloquear la ventana gráfica, grafica los ejes en tiempo real, guarda los archivos CSV y calcula métricas de red.

En el programa de la Raspberry Pi se separó la recepción de red de la interfaz gráfica utilizando un hilo en segundo plano y dos colas de comunicación. De esta manera, el hilo del servidor recibe los paquetes a alta velocidad y guarda cada fila en el archivo CSV, mientras que la ventana

principal actualiza los gráficos cada 100 milisegundos sin congelarse ni perder capacidad de respuesta.

2 Comunicación con el Sensor BMI270 y Conversión de Datos

En la placa de desarrollo utilizada, el sensor BMI270 viene integrado y soldado directamente sobre el mismo circuito impreso junto al microcontrolador ESP32-S3, por lo que no requiere cables ni conexiones externas. Internamente, las pistas de la propia placa comunican ambos chips mediante el bus I2C a una velocidad de 100 kHz (a través de las líneas internas asociadas a GPIO 42 para datos SDA y GPIO 2 para reloj SCL), utilizando la dirección esclava 0x68.

2.1 Inicialización y Configuración de Registros

A diferencia de otros sensores más simples, el BMI270 necesita que el microcontrolador cargue un archivo de configuración de 8192 bytes en su memoria interna cada vez que se enciende. Durante el arranque, el programa envía un comando de reinicio al registro 0x7E, comprueba que el identificador del integrado en el registro 0x00 corresponda al valor 0x24 y desactiva el ahorro de energía avanzado en el registro 0x7C. Luego transfiere los 8192 bytes de inicialización a través del registro 0x5E, verifica en el registro de estado 0x21 que el sensor haya quedado operativo y habilita tanto el acelerómetro como el giroscopio escribiendo en el registro 0x7D.

Una vez que el sistema está en marcha y el usuario inicia una sesión desde la Raspberry Pi, el microcontrolador actualiza la frecuencia de muestreo del acelerómetro en el registro 0x40 (seleccionando 25 Hz, 50 Hz o 100 Hz) y el rango de medición en el registro 0x41 (seleccionando ± 2 g, ± 4 g, ± 8 g o ± 16 g). Por su parte, el giroscopio se mantiene configurado a una tasa de 400 Hz con un fondo de escala de $\pm 2000\,^{\circ}/\mathrm{s}$ en los registros 0x42 y 0x43.

2.2 Lectura de Muestras y Paso a Unidades Físicas

Para obtener cada muestra, el programa consulta primero el registro de estado 0x03 para confirmar que existe un dato nuevo disponible. Cuando la lectura está lista, se leen de una sola vez 12 bytes consecutivos comenzando desde el registro 0x0C hasta el 0x17. Hacer esta lectura continua en una sola operación I2C asegura que los seis ejes (tres de aceleración y tres de giroscopio) correspondan exactamente al mismo instante de tiempo.

Cada eje se entrega en dos bytes ordenados desde el menos significativo al más significativo, formando un número entero con signo de 16 bits que va desde -32768 hasta $+32767$. Para transformar este valor crudo a unidades físicas, se multiplica la lectura por el fondo de escala activo y se divide por 32768. En el caso del acelerómetro, el resultado se expresa en metros por segundo al cuadrado ($\mathrm{m/s^2}$) considerando una gravedad estándar de $9,80665\ \mathrm{m/s^2}$:

$$a = \text{lectura}_{\text{acc}} \times \frac{\text{rango} \times 9,80665}{32768}, \qquad \omega = \text{lectura}_{\text{gyr}} \times \frac{34,90659}{32768} \tag{1}$$

donde $34,90659$ rad/s equivale a los $2000\,^\circ$/s del giroscopio convertidos a radianes por segundo. Con la placa apoyada horizontalmente en reposo, esta conversión entrega cerca de 0 m/s^2 en los ejes X e Y, y aproximadamente $9,81$ m/s^2 en el eje Z debido a la aceleración de gravedad terrestre.

3 Decisiones de Diseño en el Microcontrolador y en la Red

3.1 Modo Secuencial frente a Modo Paralelo

El sistema permite elegir desde la interfaz gráfica entre dos formas de ejecutar la captura en el ESP32-S3: modo secuencial y modo paralelo.

En el modo secuencial, una única tarea realiza todo el trabajo paso a paso: verifica si llegó una orden de detención, lee el sensor por el bus I2C, empaqueta la muestra en texto JSON, la transmite por Wi-Fi hacia la Raspberry Pi y finalmente espera hasta completar el periodo de muestreo. Aunque este esquema es sencillo y ocupa menos memoria, tiene la desventaja de que la lectura del sensor depende directamente de lo que demore el envío por red. Si la conexión Wi-Fi sufre un retraso momentáneo al enviar un paquete, la siguiente lectura del sensor se atrasa.

Para solucionar este problema se implementó el modo paralelo, que aprovecha los dos núcleos del procesador ESP32-S3 mediante una cola de memoria de 64 posiciones:

- En el núcleo 1 se ejecuta una tarea dedicada exclusivamente a leer el sensor BMI270 al ritmo exacto de la frecuencia configurada (cada 40 ms, 20 ms o 10 ms). Apenas obtiene una lectura, le asigna su marca de tiempo en milisegundos y la guarda en la cola sin esperar a la red.
- En el núcleo 0 se ejecuta la tarea encargada de las comunicaciones, que retira las muestras almacenadas en la cola y las envía por Wi-Fi a la Raspberry Pi. Si la red se demora unos instantes, la cola acumula las muestras temporalmente sin que el núcleo 1 pierda el ritmo de lectura del sensor.

3.2 Protocolo de Comunicación y Manejo de Desconexiones

Para coordinar el inicio y el fin de cada captura sin perder mensajes de control, se diseñó un esquema donde las órdenes viajan siempre por el socket TCP, mientras que los datos del sensor pueden enviarse por TCP o por UDP según lo que elija el usuario en la ventana de configuración.

Al presionar el botón de inicio, la Raspberry Pi envía una línea de texto con la frecuencia, el rango, la cantidad de muestras, el modo de lectura y el protocolo elegido. Durante la sesión, cada muestra se envía como una línea en formato JSON que contiene el número de secuencia, el tiempo en milisegundos, las tres aceleraciones, las tres velocidades angulares y la configuración activa. Al finalizar la captura (ya sea por completar las muestras solicitadas, por cumplirse el

tiempo programado o por presionar el botón de detener), el microcontrolador envía un mensaje de cierre indicando el total de muestras transmitidas.

El servidor en la Raspberry Pi utiliza la función de selección no bloqueante del sistema operativo para vigilar al mismo tiempo el socket TCP y el socket UDP. Además, cada muestra recibida se vuelca inmediatamente al archivo CSV en disco; así, ante un corte de energía o una desconexión inesperada del Wi-Fi, los datos capturados hasta ese instante no se pierden y el servidor vuelve a quedar a la espera de que el ESP32-S3 se reconecte automáticamente.

4 Resultados y Análisis Experimental

4.1 Efecto de la Frecuencia de Muestreo y del Rango sobre los Datos

Para evaluar el comportamiento del sensor se realizaron pruebas variando tanto el rango de sensibilidad (±2 g, ±4 g, ±8 g y ±16 g) como la frecuencia de muestreo (25 Hz, 50 Hz y 100 Hz). Dado que el conversor analógico-digital del BMI270 tiene una resolución fija de 16 bits para todos los rangos, cambiar el fondo de escala modifica directamente el tamaño del paso mínimo que el sensor puede distinguir, como se muestra en la Tabla 2.

Tabla 2: Relación entre el rango seleccionado, la sensibilidad del sensor y el efecto observado en las pruebas.

Rango	Sensibilidad (LSB/g)	Paso (mm/s^2)	Comportamiento Observado en los Datos
±2 g	16384	0,60	Permite distinguir inclinaciones muy leves y vibraciones pequeñas con gran detalle, pero al mover o sacudir la placa con la mano la señal alcanza rápidamente el tope de $19,61$ m/s^2 y los picos quedan recortados.
±4 g	8192	1,20	Ofrece un buen equilibrio para movimientos manuales moderados, midiendo con claridad la gravedad en reposo sin saturarse ante desplazamientos normales.
±8 g	4096	2,39	Captura sacudidas rápidas y pequeños golpes sobre la mesa (hasta $78,45$ m/s^2) conservando la forma real de los picos de aceleración.
±16 g	2048	4,79	Evita cualquier recorte incluso en impactos bruscos (hasta $156,9$ m/s^2), aunque tiene ocho veces menos resolución fina que el rango de ±2 g para movimientos suaves.

Por otro lado, al comparar las tres frecuencias de muestreo se observa un compromiso directo entre el nivel de detalle temporal de la señal y la cantidad de datos generados por segundo:

- A 25 Hz se toma una muestra cada 40 milisegundos. Esta velocidad es suficiente para registrar giros lentos o cambios de inclinación de la placa generando archivos livianos, pero cuando se producen golpes o sacudidas rápidas que duran menos de 20 milisegundos,

el pico máximo ocurre entre dos lecturas y no alcanza a registrarse en el gráfico ni en el archivo CSV.

- A 50 Hz el intervalo entre muestras baja a 20 milisegundos. Con esta tasa la gráfica en tiempo real se observa mucho más continua y se logra representar bien el movimiento normal de la mano al agitar o rotar el sensor.
- A 100 Hz el sensor se lee cada 10 milisegundos, lo que permite reconstruir con fidelidad impactos breves y cambios rápidos de dirección. En esta frecuencia se nota además la ventaja del modo paralelo: como 10 milisegundos es un margen muy estrecho, cualquier demora del Wi-Fi en el modo secuencial retrasa la lectura siguiente y baja la frecuencia efectiva, mientras que el modo paralelo mantiene el muestreo estable cerca de los 100 Hz configurados.

4.2 Comparación Experimental entre TCP y UDP

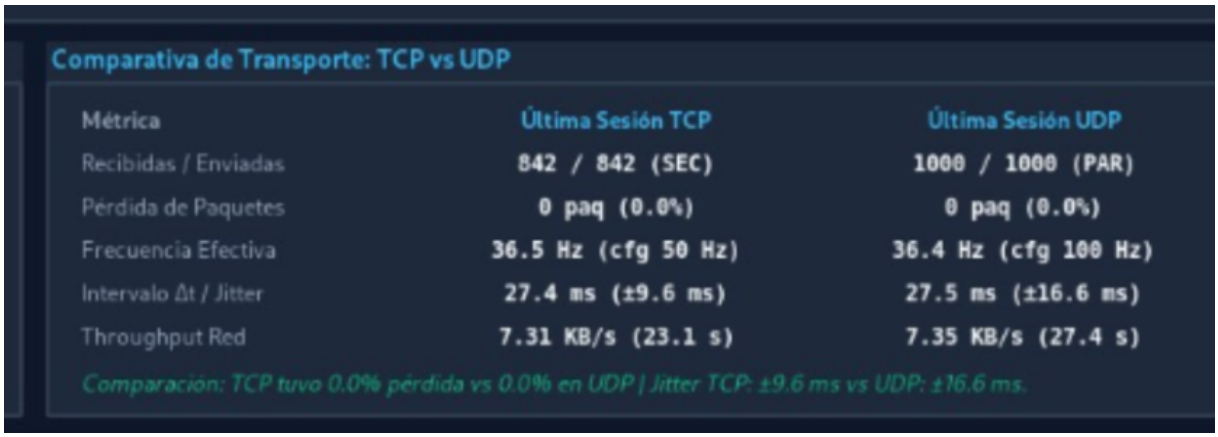
Finalmente, se comparó la transmisión de las muestras utilizando los protocolos TCP y UDP en el puerto 5000, apoyándose tanto en la tabla comparativa integrada en la interfaz gráfica como en la inspección de paquetes. Los resultados principales de esta comparación se sintetizan en la Tabla 3.

Tabla 3: Comparación del comportamiento de la transmisión de datos entre TCP y UDP.

Aspecto Evaluado	Protocolo TCP	Protocolo UDP
Confirmación de paquetes	Cada envío de datos desde el ESP32-S3 requiere recibir una confirmación (ACK) desde la Raspberry Pi.	El ESP32-S3 envía cada muestra como un datagrama independiente sin esperar confirmaciones de recepción.
Sobrecarga en la red	Mayor cantidad de tráfico debido a las cabeceras de 20 a 32 bytes y a los paquetes de confirmación en sentido inverso.	Menor sobrecarga, con una cabecera fija de solo 8 bytes y tráfico únicamente desde el microcontrolador hacia el servidor.
Integridad de los datos	Garantiza que el 100% de las muestras lleguen en orden al archivo CSV, retransmitiendo automáticamente si un paquete se pierde.	No retransmite paquetes perdidos; si hay interferencia en el Wi-Fi, la muestra se omite y el sistema lo detecta por el salto en el número de secuencia.
Regularidad temporal	Presenta mayor variación en el tiempo de llegada entre muestras cuando ocurren retransmisiones en la red.	Entrega las muestras con un intervalo de llegada más parejo y menor latencia, favoreciendo el graficado fluido en tiempo real.

Para comparar el comportamiento de los protocolos TCP y UDP, se utilizaron las estadísticas registradas por la interfaz gráfica desarrollada en la Raspberry Pi. En ambas sesiones se observó un 0 % de pérdida de muestras según los registros de la aplicación. TCP alcanzó una frecuencia efectiva de 36,5 Hz y un jitter reportado de ±9,6 ms, mientras que UDP alcanzó 36,4 Hz y un jitter de ±16,6 ms.

Aunque UDP permite transmitir datos sin esperar confirmaciones del protocolo de transporte, los resultados obtenidos no muestran una mejora en la frecuencia efectiva respecto de TCP. Además, es importante considerar que las pruebas se realizaron con distintas frecuencias configuradas, por lo que los resultados no permiten establecer una comparación directa del rendimiento de ambos protocolos en condiciones equivalentes.



Comparativa de Transporte: TCP vs UDP

Métrica	Última Sesión TCP	Última Sesión UDP
Recibidas / Enviadas	842 / 842 (SEC)	1000 / 1000 (PAR)
Pérdida de Paquetes	0 paq (0.0%)	0 paq (0.0%)
Frecuencia Efectiva	36.5 Hz (cfg 50 Hz)	36.4 Hz (cfg 100 Hz)
Intervalo Δt / Jitter	27.4 ms (±9.6 ms)	27.5 ms (±16.6 ms)
Throughput Red	7.31 KB/s (23.1 s)	7.35 KB/s (27.4 s)

Comparación: TCP tuvo 0.0% pérdida vs 0.0% en UDP | Jitter TCP: ±9.6 ms vs UDP: ±16.6 ms.

Figura 1: Comparativa de Transporte entre TCP y UDP.

5 Conclusiones

El sistema desarrollado permitió integrar de manera completa la lectura por hardware del sensor inercial BMI270 en el microcontrolador ESP32-S3 con un servidor multiprotocolo y una interfaz gráfica en la Raspberry Pi 5. La estructura cliente-servidor implementada cumple con todos los requerimientos del proyecto, permitiendo modificar desde la interfaz la frecuencia de muestreo, el rango de aceleración, la modalidad de duración, el modo de ejecución y el protocolo de red sin necesidad de reprogramar el microcontrolador.

En cuanto al ajuste de los parámetros del sensor, las pruebas demostraron que la elección del rango y de la frecuencia depende del tipo de movimiento que se desea estudiar. Los rangos pequeños (± 2 g y ± 4 g) aprovechan mejor las 32768 cuentas del conversor para medir inclinaciones suaves con alta precisión, pero se saturan ante movimientos rápidos, donde resulta necesario utilizar los rangos de ± 8 g o ± 16 g. De forma complementaria, elevar la frecuencia de muestreo de 25 Hz a 100 Hz resulta fundamental para registrar impactos breves que de otro modo pasan desapercibidos entre dos lecturas sucesivas.

Por último, la implementación del modo paralelo en los dos núcleos del ESP32-S3 demostró ser una mejora importante frente al modo secuencial al trabajar a 100 Hz, ya que el uso de una cola en memoria independiza el reloj de lectura del sensor de las demoras propias del Wi-Fi. Asimismo, la comparación entre protocolos permitió observar diferencias en el comportamiento. Notamos que TCP proporciona una transmisión fiable y ordenada de los datos, mientras que UDP utiliza un mecanismo de comunicación más sencillo, sin confirmaciones de entrega a nivel de transporte. En las pruebas realizadas, ambos protocolos registraron un 0 % de pérdida de muestras, con frecuencias efectivas similares. Sin embargo, TCP presentó una menor variación

temporal entre las muestras recibidas, de acuerdo con los valores de jitter reportados por la aplicación.